

程多多 Java 与 Agent 应用工程师面试问答

马素超面试备考手册

本手册用于准备程多多商旅方向的 Java 与 Agent 应用开发面试。你的主线是 Java 后端工程能力、Agent 实习经验和可展示的 homegrown 项目。回答时先说业务目标，再讲自己的实现、关键取舍和失败处理，最后说明如何迁移到商旅业务。

先练这六个问题

1. 用两分钟介绍 homegrown，并解释哪些决策由代码负责，哪些交给模型。
2. 用户说“下周三去北京出差两天”，你怎样设计一个真正能完成预订的 Agent。
3. 下单接口超时了，怎样避免重试造成两张机票或两笔订单。
4. 简历中的 RAG、10+ Skill、简化 FSRs，分别对应当前哪个模块或哪个历史版本。
5. 文献检索项目准确率提升约 23%、Token 减少 21%，分母、基线和评测方法是什么。
6. 一个 LLM 请求越来越慢，你怎样区分模型耗时、重试放大、线程阻塞和数据库连接耗尽。

面试官可能怎样评估你

JD 同时要求 Java 基础、后端稳定性、Agent 工具调用、记忆、RAG 和效果评测。由此推测，技术面会重点检查你能否把模型接入现有业务系统，能否处理不确定输出与确定性业务规则之间的冲突。你的全栈经历有助于说明交付能力，但技术回答应把后端与 Agent 放在前面。

使用方式

每题先用 60 至 90 秒口述“参考回答”，再练“追问”。P0 为优先准备题，P1 为常见深入题。项目问题中的“当前实现”指本次查看的 homegrown 工作区代码；“设计题”中的方案是拟议实现，不能当作已上线业绩。其他项目与实习的参考回答以简历描述为基础，个人职责、时间、指标需要与你能展示的材料一致。

阅读导航

第 2 至 3 页：公司与简历核对。第 4 至 11 页：homegrown 项目深挖。第 12 至 16 页：Agent、RAG 与商旅系统设计。第 17 至 21 页：Java、Spring、数据库与工程化。第 22 至 23 页：其他项目、实习与沟通。第 24 至 25 页：现场题、模拟面试和准备清单。第 26 页：大模型基础补充。第 27 页：来源与代码定位。

编写日期 2026 年 9 月 9 日。面试关注点为根据岗位与业务的推测，不代表公司真实题库。

01 公司业务与岗位判断

01 P0 你为什么想来程多多

考察点：是否了解企业商旅业务，求职动机是否能落到岗位。

参考回答：我关注这个岗位，是因为它同时要求 Java 后端和 Agent 应用开发，和我的面试备考平台及 Agent 实习经历比较契合。程多多的产品介绍涉及企业差旅申请、机票和火车票预订、行程管理以及差旅政策管控。我理解这里的 AI 价值包括减少需求沟通和重复操作，并且让结果符合企业规则。我的项目里也遇到过类似问题：模型适理解与生成，状态推进和权限必须由服务端控制。我希望把这类工程经验用于更具体的业务流程，先从查询推荐或规则问答开始，再逐步接入审批和预订。

追问：你了解我们的哪些内部技术？**答：**目前了解的是公开产品能力和 JD，内部技术架构、供应商接口与 Agent 阶段，需要入职后和团队确认，不会把行业通用方案当成贵司现状。

02 P0 企业商旅和个人旅游预订有什么不同

考察点：是否看到支付者、使用者、审批者并不相同。

参考回答：企业商旅里，员工是出行人，企业承担费用，管理者关注差标和审批，财务关注对账结算。一个“最便宜”的方案也可能不合适，例如到达太晚、退改成本高，或者不在企业协议范围内。因此查询时既要满足行程需求，也要检查企业、员工职级、城市和日期对应的规则；超标时要解释原因并走例外审批。预订后还要跟踪出票、行程变化、退款和账单，才能完成闭环。技术上我会重点考虑企业数据隔离、规则版本、订单状态一致性和全流程审计。

追问：降本怎么衡量？**答：**要用同条件下可比的含税总价、退改费用与服务成本衡量，也统计申请到完成的耗时、人工接管率和结算差异率。不能仅看票面价格。

03 P1 你认为这个岗位的面试官最关心什么

考察点：是否能从 JD 提炼交付目标。

参考回答：我的判断是，首先要有可靠的 Java 基础，能够参与商旅后台开发；其次，要能把自然语言需求转换成可校验的工具参数和任务步骤；最后，要证明系统可运行、可排错、可评测。面试官可能会从我的项目问到多轮上下文、异步调用、模型输出校验，再转到下单幂等、异常恢复和差标控制。我会优先讲能定位到代码的实现，并说明复杂业务接入时需要补哪些能力。

业务依据：程多多官网 <https://www.chengduoduo.com/> 与开发者发布的 App Store 产品介绍

<https://apps.apple.com/cn/app/程多多/id1506135728>。官网强调企业 OA 对接与结算服务；上述面试侧重点为推测。

02 简历表述与当前代码核对

面试前必须对齐的六处表述

RAG 链路：简历写了 pgvector、查询改写、TopK 和相似度阈值。当前源码可以定位到资料解析、章节切分、标注和上下文注入；V29 中有 vector_store、1024 维向量和 HNSW 索引定义，但当前依赖与 Java 主链路中没有找到对应的向量化与检索调用。表存在不能证明完整向量 RAG 正在运行。准备好历史提交、运行截图或另一 RAG 项目的代码，说明版本边界。

10+ Skill：当前 application.yml 明确列出 6 个 interview.skills 配置项，InterviewSkillService 负责读取这些方向配置。若 10+ 指历史版本或其他模块的模板，需要列清名称、配置文件和调用入口，避免把“技术方向配置”讲成“自主工具编排系统”。

FSRS：ScheduleService 当前采用 AGAIN、HARD、GOOD、EASY 的固定天数规则，并考虑是否计时、是否引导和是否看答案。可以称为借鉴 FSRS 反馈形式的简化间隔复习；不能声称已实现完整 FSRS 的稳定性、难度与可提取性公式，也不要沿用 README 其他模块的间隔表作为此服务的结果。

掌握闸门：V28 的部分唯一索引是对 user_id 建立，条件是 READY 或 ANSWERING。因此约束是“每个用户最多一个这两种状态的 run”，不是“每人每题一个”，也不自动覆盖 GUIDED 等其他阶段字段。

持久化：模拟面试基础题存 PostgreSQL，进行中的问答流使用 Redis，TTL 为 24 小时，回答在结束或评估路径写入关系库。不能保证任何时刻 Redis 丢失后都能完整恢复正在进行的对话。

模型调用：StructuredOutputInvoker 的实际 rawOrSpring 方法先走原生客户端，原生不可用或返回空后再考虑 Spring AI。部分注释写的是另一顺序，面试应以方法执行路径为准。不要把项目的兼容处理概括成所有模型或所有 Spring AI 版本的缺陷。

可以怎样诚实说明

“这份简历概括了项目的演进能力。当前版本我最能完整演示的是资料结构化、学习闭环、模型适配和 SSE。向量检索以及 Skill 数量，我会按对应版本展示，不把历史能力混入当前主链路。文献 RAG 项目则单独解释 embedding、混合召回和重排。”

哪些数字不要临场编

不要临场编造日活、QPS、平均响应时间、评测集样本量、成本节省和个人代码占比。没有正式测量时，讲已经观察到的现象、采用的机制和计划如何验证。简历中的量化成果要能说明计算口径并拿出记录。

代码依据：app/build.gradle、application.yml、StructuredOutputInvoker.java、CorpusIndexer.java、ScheduleService.java、InterviewSessionService.java、V28 与 V29 迁移。详见末页代码导航。

03 自我介绍与项目总览

04 P0 请做一个一分钟自我介绍

考察点：能否迅速建立与岗位的关联。

参考回答：您好，我叫马素超，目前是上海理工大学电子信息专业的 2027 届硕士，本科是软件工程。我主要做 Java 后端和 Agent 应用开发，也能完成 React 前端和容器化交付。最近重点做的是 AI 面试备考平台 homegrown，后端使用 Java 21、Spring Boot 和 PostgreSQL，支持简历分析、资料学习、模拟面试和多 Provider 模型接入。我比较关注模型应用的工程问题，例如状态机如何控制流程、结构化输出失败怎样处理、流式请求怎样中断，以及用户数据怎样隔离。我的实习还接触过 LangChain、LangGraph 的工具调用、任务路由和记忆管理。这个岗位把 Java 业务开发和商旅 Agent 结合起来，我希望重点介绍这些相关经验。

追问：能来现场吗、每周几天、持续多久？**答：**明确说出你真实可到岗日期、每周出勤天数及可持续月份；提前与学校安排确认。JD 明确写了需到场，不能只回答“时间比较灵活”。

05 P0 用两分钟介绍 homegrown

考察点：是否能讲清问题、架构、贡献和边界。

参考回答：homegrown 是面向学习和模拟面试的应用。它要解决的主要问题是，普通对话很难持续记录学习进度，出题容易重复，评价标准也不稳定。我把系统做成模块化单体，主要业务域包括用户、简历、项目解析、知识库、学习训练和模拟面试。Java 服务端管理知识点、练习状态、评分结果与复习安排；模型按任务生成题目、讲解、追问或结构化分析。前端用 React 和 TypeScript，长文本通过 SSE 输出，同一套界面也被 Electron 复用。项目里我最值得展开讲的是确定性流程控制、多模型兼容和流式体验。生产级的高可用、多租户配额以及完整效果评测还需要进一步建设。

追问：你个人做了什么？**答：**按可展示的提交和模块说明“由我负责、共同参与、仅接入调用”的范围，选择自己真正能解释的两条调用链，不用“所有模块都很熟悉”代替证据。

06 P0 为什么采用模块化单体

考察点：是否根据规模选择架构。

参考回答：这个项目业务变化快、模块之间联系紧密，单体更方便验证闭环和做本地交付。模块按业务域划分，每个域里再分 Web、Service、Repository 和模型层，共享 AI、文件与异常处理设施。这样减少了服务发现、跨服务事务和多套部署的负担。如果未来资料索引与在线对话负载差异明显，我会先把索引任务作为独立 Worker 拆出去，再根据实际团队边界和负载决定是否拆更多服务。

追问：怎么避免单体变成“大泥球”？**答：**明确模块 API 和数据归属，限制跨模块直接操作 Repository；对关键依赖做架构约束检查。单体不等于可以随意跨域读写。

04 学习流程与确定性控制

07 P0 你的项目算 Agent 吗 和工作流有什么区别

考察点：是否理解自主性及其适用边界。

参考回答：我会把当前 homegrown 描述为由服务端工作流主导的 AI 应用，部分环节具有基于反馈继续追问的行为。它的核心调度不是让模型自由决定，而是代码先选学习内容和状态，再调用模型生成。通常 Agent 会根据目标和工具结果动态选择下一步，工作流则有更明确的执行路径。商旅里，我倾向让 Agent 理解需求、澄清参数和推荐方案，而把差标校验、审批条件、订单提交资格放在业务服务中。自主程度要随动作风险和结果可验证程度调整。

追问：为什么不让大模型决定全部流程？**答：**业务规则需要稳定执行，模型可能受上下文、提示注入和随机输出影响；即使提示词要求遵守，也应在执行工具前再次校验。

08 P0 你怎样保证用户不会跳过当前练习

考察点：状态机是否真正约束后端。

参考回答：我把一次训练建模成 DrillRun，后端依据状态和训练阶段判断可执行动作。数据库里还通过 V28 的部分唯一索引限制每个用户最多一个 READY 或 ANSWERING 的 run。应用层先检查状态，数据库兜住并发创建。但这个索引只覆盖指定状态，不能单独代表整个学习流程的约束。对评分、引导和完成动作还要逐个验证前置状态，并处理重复提交和并发状态变化。

追问：两个请求都检查通过怎么办？**答：**唯一索引能兜住创建竞争；更新状态则可以用版本号或带旧状态条件的 UPDATE，并检查受影响行数。唯一约束异常要在合适的事务边界外转换成“已有进行中任务”，不要在已回滚标记的事务里继续写。

09 P1 你实现的是完整 FSRS 吗

考察点：能否区分借鉴思想与算法复现。

参考回答：当前 ScheduleService 是简化规则，并非完整 FSRS。它把反馈分成四档，AGAIN 或看过答案时当天再练；HARD 是一天；GOOD 和 EASY 根据计时等条件给较短或较长间隔，引导后的 GOOD 会缩短。完整 FSRS 需要持续更新记忆稳定性、难度等状态，并基于目标保留率推算间隔。我现在优先实现可解释的复习闭环。后续如果有足够复习日志，会接入完整实现，比较到期回忆率和单位时间学习收益。

追问：GOOD、EASY 由模型决定会不会漂移？**答：**模型输出证据和要点评价，分数聚合及评级规则由代码控制；还需要固定评测集检验不同模型和提示词版本的偏差。

05 选题去重与评分

10 P1 到期优先选题怎样实现

考察点：是否理解数据范围和业务优先级。

参考回答：SelectionService 会结合知识点与用户掌握记录选题，重点是先把候选范围收窄到当前学习方向，再选到期复习或低层未掌握内容。否则即使到期排序正确，也可能给用户推送另一个方向的题。DailyPlanService 则为方向生成每日复习与新学任务，并提前准备题目。我会把“候选范围、优先级、配额、结果状态”拆开解释，这样更容易写边界测试，例如方向为空、没有到期项、任务已完成和重复触发。

追问：数据量上来会慢在哪里？答：当前部分逻辑先读集合再在 Java 中筛选，规模变大时要把过滤、排序和分页下推数据库，围绕 user_id、plan_id、due_at 等访问模式建立索引，并通过执行计划验证。

11 P1 N gram 去重为什么不用向量

考察点：是否清楚成本、效果和失败案例。

参考回答：当前用字符级 trigram。先统一大小写并去掉空白和标点，再把文本切成连续三个字符的集合，用交集大小除以并集大小得到 Jaccard 相似度。它计算可重复、无需外部服务，适合挡住字面重复和少量换词的题目。局限是抓不住表达差异很大的同义题，也可能把同一技术主题下两道不同题判得过近。SimilarityGuard 抽象了接口，后续可以用语义向量补充，而不是直接替代所有规则。

追问：阈值怎样定？答：准备“重复、相似但不同、完全不同”的人工标注题对，观察误杀与漏放，再选阈值。不存在对所有题型通用的最佳阈值。

12 P0 如何避免模型乱评分

考察点：是否知道输出格式正确不等于评价正确。

参考回答：我把评价拆成模型判断和代码汇总。GraderText 结合标准要点与实际问答实录，让没有问到的点标记为 NA，避免拿未提问的追问要求扣分；GradeScale 再汇总要点评价。学习流程也区分独立回答与引导后回答，记录初始评价和最终评价，引导后通过不会直接当作最高档。看过答案的场景还要保留揭示边界，避免把照抄当作独立掌握。剩下的关键工作是构建人工标注样本，检查同义答案、关键错误与诱导评分等边界。

追问：temperature 设成 0 就能稳定吗？答：不能保证完全可复现。还要控制模型版本、Prompt、上下文、规则版本，必要时缓存已完成评价，并评估人工与模型的一致性。

06 模型适配与输出校验

13 P0 多 Provider 接入到底难在哪里

考察点：是否真的处理过协议差异。

参考回答：OpenAI Compatible 通常表示主要请求结构相近，不保证所有参数和流式字段完全一致。项目同时保留 Spring AI ChatClient 与 JDK HttpClient 通道，集中处理 base URL、模型名、参数、结构化转换与流式解析。当前 StructuredOutputInvoker 实际优先调用原生客户端，再在条件允许时回退 Spring AI。适配层会按 Provider 处理可选思考参数，对部分 400 错误做去参重试。下一步我会把按名称推断能力整理为明确的能力配置和契约测试，减少分支猜测。

追问：400 都应该重试吗？答：不应该。只有明确是可选参数不支持且仍保留必要语义时才降级；认证、模型名错误、上下文过长要分别处理。项目现有按状态码去参的做法仍偏宽，需要收紧。

14 P0 模型返回 JSON 为什么还会失败

考察点：是否掌握结构化输出的多层验证。

参考回答：模型可能返回代码围栏、解释文字、截断 JSON、错误类型，或者返回合法但不符合业务的内容。项目用 BeanOutputConverter 提供结构要求，并清理围栏、提取 JSON，解析失败后把错误带入有限次数重试。这个机制主要解决语法和映射问题。业务上还应校验必填字段、枚举、范围、引用 ID 和数量，例如生成了不存在的知识点 ID，即使能反序列化也不能入库。涉及订单时，必须由业务服务重新核验用户身份、库存和授权。

追问：让模型修复一次失败输出是否等于安全？答：不是。修复结果仍是外部输入，仍须全套校验；连续失败时明确返回失败并记录原因，不能用随意默认值继续交易。

15 P1 reasoning 字段能直接当正文吗

考察点：是否理解兼容兜底带来的语义风险。

参考回答：当前客户端存在正文为空时读取兼容 reasoning 字段的路径，也允许一些流式调用关闭该行为。这是具体接入场景的兜底，不应该默认视为所有模型都可安全使用。推理字段可能包含中间猜测而不是最终答案，因此我会优先遵守 Provider 的公开输出契约；对结构化任务检查内容确实符合目标 Schema，对面向用户的正文只展示明确的最终输出或受支持的摘要。正文缺失时可以明确失败或切换经过验证的通道。

追问：怎样证明兼容性？答：为每个支持的 Provider 保存脱敏请求与响应样例，覆盖正常、空正文、分片、结束标记、参数错误与中断，运行契约回归。

07 重试预算与异步上下文

16 P0 一个请求为什么可能重试很多次

考察点：是否会分析分层重试放大。

参考回答：如果业务层、SDK 和网关都重试，请求次数会叠加甚至相乘。项目的 Provider 注册代码把 SDK 的 `maxRetries` 设为 0，把结构化重试集中在外层；但原生客户端的参数降级和通道回退也会增加真实调用，所以外层两次不一定等于总共两次 HTTP 请求。我会按请求 ID 记录每次尝试的通道、原因、耗时和 Token，设置总 deadline、最大尝试次数和费用预算。重试只针对可恢复错误；非幂等的业务工具不能沿用文本生成的重试策略。

追问：429 怎么办？**答：**结合 `Retry-After` 和指数退避加抖动，设置并发与速率上限。预算不足时排队或拒绝；不要让每一层独立重试。

17 P0 异步任务如何拿到用户模型配置

考察点：线程上下文是否会丢失或串用户。

参考回答：请求中的用户身份和模型配置不应该假设会自动进入线程池。`CorpusIndexer` 在提交前取得当前请求的配置快照，然后用 `AiSettingsService.withTaskConfig` 在任务线程里设置上下文，结束后在 `finally` 中恢复或移除 `ThreadLocal`。这避免任务运行时从另一个请求取配置，也防止线程复用后残留。生产里还应把 `user_id`、`tenant_id`、`trace_id` 和任务 ID 显式传入。API Key 不写进日志，也不直接明文放入普通消息队列。

追问：把 key 放进快照安全吗？**答：**要控制生命周期与传播范围。持久任务优先存受权限保护的凭据引用，执行时读取密钥；长时间排队还应重新检查凭据和用户权限。

18 P1 资料索引为什么用单线程加有界队列

考察点：是否理解异步资源控制与可靠性。

参考回答：当前 `CorpusIndexer` 用一个工作线程、容量 64 的有界队列和 `AbortPolicy`，避免同时触发太多模型请求；`running` 集合阻止同进程内对相同资料重复提交，拒绝时标记失败并允许重试。索引时先读取和调用模型，最终写库在短事务中锁住资料行，避免用户已删除资料后写回孤儿数据。这个方案适合当前规模，但队列在进程内，重启会丢失待执行任务，`running` 也不跨实例。

追问：怎么升级？**答：**数据库持久化任务状态，以租约或条件更新领取任务，失败退避、超时回收；多实例共享队列并做幂等写入。只增加线程数不能解决任务可靠性。

08 SSE 与端到端流式通信

19 P0 为什么选择 SSE 而不是 WebSocket

考察点：协议选择是否对应交互需要。

参考回答：主要需求是用户提交一次问题后，服务端持续推送模型输出，SSE 能满足这种单向事件流，并且可以沿用 HTTP 鉴权和代理。前端用 `fetch` 读取响应流，可以携带 `Authorization` 和请求体。SSE 的事件边界是空行，而网络 `chunk` 可能截断一行或包含多个事件，因此前端 `parser` 要保留缓冲区，处理 CRLF、多行 `data` 和心跳注释。项目有 `sseParser.test.mjs` 覆盖解析边界。如果以后增加实时语音、双向高频控制，我再评估 WebSocket。

追问：SSE 是否天然能续传？**答：**协议可以使用事件 ID 配合重连，但应用还要保留可重放的事件和任务状态。当前流式显示不等于完整断点续传。

20 P0 用户点击停止后模型真的停止了吗

考察点：是否理解前端取消与上游取消的区别。

参考回答：前端 `AbortController` 中止读取后，服务端还需要检测连接失效并释放上游请求。项目 `SseWriter` 每 20 秒发心跳，写失败会关闭流并中断拥有该请求的线程，客户端代码也处理部分中断与输入流关闭。但中断能否及时停止底层 I/O，要以实际路径和测试结果判断；前端不再展示并不能证明供应商停止计算或计费。我会核对取消后后台线程、连接数和任务状态是否回落，以及取消是否被误当成可重试异常。

追问：生成一半的内容怎么存？**答：**区分 `COMPLETED`、`CANCELLED`、`FAILED`，保留已生成内容时带上状态，不把部分答案缓存为完整结果；涉及业务执行的任务还要独立查询真实状态。

21 P1 为什么经过 Nginx 后流式变成一次性输出

考察点：能否排查完整链路。

参考回答：先对比直连后端与经过代理时的首字节和分块时序。如果只有代理链路聚合，要检查代理缓冲、压缩、超时及中间网关。项目 `nginx.conf` 设置了 `proxy_buffering off`，服务端按事件 `flush`，流中保留心跳。当前超时配置较长，这是兼容长生成的取舍；商旅交互还需要更短的业务截止时间和可取消任务，不能无限等待。另一个常见问题是前端分帧错误，收到字节但没有触发事件，也会表现为“不流式”。

追问：HTTP 已经返回 200 后出错怎么办？**答：**发送约定的 `error` 事件并结束流；前端区分失败和正常完成，不能指望再修改 HTTP 状态码。

09 Redis 会话与并发提交

22 P0 Redis 里的面试会话过期会发生什么

考察点：是否知道缓存和事实数据的区别。

参考回答：当前基础题已经存入 PostgreSQL，进行中的问答列表和完成标记仍依赖 Redis，问答 key 的 TTL 是 24 小时。回答在结束或评估路径批量写入数据库。如果尚未写库的数据丢失，仅靠基础题和会话表不能还原所有回答。我会把每次用户回答及下一题状态以短事务先落库，Redis 保存可重建的视图；重连时从数据库恢复。这样会增加写入次数，但对需要可靠恢复的商旅任务很有必要。

追问：TTL 到期和 Redis 故障是同一问题吗？答：不同。过期是生命周期策略，故障是可用性问题。延长 TTL 不能解决故障丢数据，也不能代替持久化和恢复逻辑。

23 P0 用户连续点两次提交会怎样

考察点：是否只依赖前端禁用按钮。

参考回答：前端禁用只能改善体验，后端仍要防重复。当前模拟面试存在读取 Redis 整个问答列表、修改再写回的路径，如果同一会话两个请求并发，就有重复推进或覆盖更新的风险，事务也不能自动保护 Redis 的读改写。我会给每次提交带 request_id 和 turn_id，用数据库唯一约束记录已处理请求，以带版本号的条件更新推进会话。对同一会话的模型生成可以串行化，但最终状态必须有持久约束。

追问：直接用分布式锁就够了吗？答：还要考虑锁过期、进程暂停和请求超时。锁可减少竞争，幂等键与条件状态更新负责最终正确性；不能只靠锁保证不会重复下单。

24 P1 追问怎样做到既自然又可控

考察点：是否能结合模型能力与业务限制。

参考回答：当前基础题由服务端组织，第一题固定为自我介绍，技术题由模型生成；回答后由 FollowupGeneratorService 结合主问题、实际回答和难度生成追问。服务端控制追问次数和推进。对于明显表示不会的回答，先做基础确认，再决定结束追问。当前 looksIgnorant 使用长度和关键词，是可解释但较粗糙的启发式，可能误判“我没做过但理解原理”这类回答。可以增加结构化意图判断，同时保留轮数与时间上限。

追问：商旅里能复用什么？答：复用“模型生成下一问，服务端决定能否继续”的模式；必须收集的行程参数由 Schema 和校验规则判断，不用回答长度判断是否可以下单。

10 文件知识库与项目安全

25 P0 你的资料处理链路具体怎么走

考察点：是否能从入口讲到数据落地。

参考回答：资料摄取后保存原文和元数据，再交给 CorpusIndexer 做结构化索引。当前切块按 Markdown 或章节标题、段落边界进行，软上限是 6000 字符，长块继续切分；超过块数限制会合并尾部，所以最终某一块仍可能很大。模型负责标题、主题、摘要和总览标注，原文由代码保存，标注失败时回退基础索引。这是当前能定位到的主链路。向量化、查询改写与 TopK 检索应按独立版本或文献 RAG 项目展开，不能由 pgvector 建表推导为已启用。

追问：这种切块能直接用于商旅 RAG 吗？答：规则手册还要保留条款层级、适用范围与版本。较大的章节块适合展示，检索可建立更小的子块并回溯父章节，避免上下文超限和规则断章取义。

26 P0 你怎样保证用户只能访问自己的数据

考察点：是否区分认证与对象权限。

参考回答：JWT 校验后从 token 的 subject 取得 user_id，不能信任请求里随意提交的身份。服务层还要检查资源归属，例如 InterviewSessionService 的 requireOwned。登录成功只证明是谁，并不代表能读取任何 sessionId 或文件。商旅扩展时要把 tenant_id、角色与数据范围纳入查询和工具校验，文件下载也应走有权限的接口或短期签名链接。缓存键同样带上隔离范围，防止跨用户命中。

追问：你当前安全配置还有什么要检查？答：现有配置对部分资料下载路径放行、兜底链允许未匹配路径访问，必须结合 Controller 的实际鉴权方式逐项验证。应通过越权测试确认，不能只看 SecurityConfig 就断言安全或泄露。

27 P1 如果面试官让你做一次上线前检查

考察点：是否对部署后的系统负责。

参考回答：我会先检查生产环境的密钥、连接地址、鉴权边界与日志脱敏，然后验证数据库迁移和产物版本。当前 Flyway 配置有关闭校验和允许乱序的开发兼容设置，正式环境需要修复历史后恢复严格校验；JWT 有较长有效期，也应设计撤销或 tokenVersion 机制。其次演练模型超时、Redis 不可用、对象存储失败和流式取消。最后确认备份恢复、健康检查、资源限制和回滚步骤。没有测过的故障恢复能力不会直接写进 SLA。

追问：本次检查能证明线上情况吗？答：源码与配置检查只能证明设计及潜在风险，实际部署环境、运行参数和故障表现需要部署证据或受控验证。

11 Agent 框架与工具调用

28 P0 一次 Tool Calling 是怎么完成的

考察点：是否清楚模型与程序各自做什么。

参考回答：应用把可用工具的名称、用途和参数 Schema 提供给模型。模型返回工具名、调用 ID 和参数；程序先校验类型、必填项、权限和业务前置条件，再执行工具，把结果按调用 ID 回传给模型，模型继续回答或选择下一步。工具描述要消除歧义，比如查询报价与创建订单分开，查询工具不能产生交易副作用。执行循环必须设置最大轮次、总耗时和预算，并把每一步结果关联到任务 ID。

追问：模型调用了不存在的工具怎么办？答：工具必须来自服务端注册表。未知工具拒绝执行，返回可恢复的结构化错误；不能根据模型返回的字符串反射调用任意方法。

29 P0 LangChain LangGraph 和 Spring AI 你怎么选

考察点：是否理解框架职责而非只记名称。

参考回答：LangChain 适合组合模型、检索器和工具等应用组件；LangGraph 更适合显式状态、分支、循环、检查点和人工介入的执行流程。Spring AI 便于在 Java 业务系统里接入模型与相关能力。我的 homegrown 以 Java 服务端工作流为主，实习中的任务路由与多 Agent 经历对应 Python 生态。商旅业务已有 Java 主系统时，我会先评估 Java 内部编排能否覆盖需求；复杂流程如使用独立 Python 编排服务，也要通过明确 API 对接，交易规则仍由业务服务负责。

追问：LangGraph 的检查点是不是数据库事务？答：不是。检查点保存执行状态，外部工具的副作用仍需要幂等与补偿。恢复时可能重新运行某个节点，因此不能把扣款直接放在可重复执行的无保护逻辑里。

30 P1 多 Agent 一定比单 Agent 强吗

考察点：是否会控制架构复杂度。

参考回答：不一定。多个 Agent 适合职责清晰、可以独立并行的工作，例如机票和酒店候选查询，或者让独立校验者检查方案。但它会增加模型调用、共享状态冲突和错误传播。单个 Agent 加几个确定性工具能完成时，我会先用更少的组件。若采用多 Agent，需要定义输入输出、最终决策权、超时和冲突处理；报价查询可并行，订单提交应由唯一的业务执行入口协调。

追问：怎样判断多 Agent 有收益？答：在同一任务集上比较任务成功率、延迟、成本与错误类型。只看回答更长或过程更复杂不能说明收益。

框架机制参考：LangGraph Persistence <https://docs.langchain.com/oss/python/langgraph/persistence> 与 Interrupts <https://docs.langchain.com/oss/python/langgraph/interrupts>。

12 记忆管理与 Prompt 设计

31 P0 你会怎样设计商旅 Agent 的记忆

考察点：是否能区分对话历史、业务事实和偏好。

参考回答：我会分三类保存。会话历史保留近期对话和工具结果，支持理解“改成明天下午”；任务状态保存已确认的城市、日期、出行人、预算、报价与审批状态，这是结构化数据；长期偏好保存常用航司、座位或酒店习惯，并标注来源、更新时间和适用用户。Redis 可以加速会话读取，任务事实应持久化。每次执行前以当前订单和规则服务为准，过期偏好不能覆盖新的明确指令。

追问：摘要压缩会丢信息怎么办？答：时间、金额、身份、确认记录等关键字段独立存储，摘要只帮助语言理解。对影响预订的歧义重新确认，不靠摘要猜。

32 P0 你如何调 Prompt 并判断是否变好了

考察点：是否有可重复的优化过程。

参考回答：先把任务和失败类型分清：是理解错需求、漏槽位、选错工具、参数不合法，还是依据不足。Prompt 中明确目标、输出格式、允许的工具和缺信息时的澄清方式，示例重点覆盖易错案例。每次改动留版本，用相同模型和固定测试集对照，观察工具选择准确率、参数校验通过率、追问次数和任务完成率。不能只凭几次对话更自然就判断提升。对于明确的差标和身份约束，我会落在代码与工具层。

追问：可以让模型自评吗？答：可以作为辅助，但要与人工标注或确定性规则校准，避免同一个模型同时生成和评价时偏袒自己的表达。

33 P0 如何防止检索文档里的提示注入

考察点：是否把外部文本当作不可信数据。

参考回答：检索片段、网页、简历以及工具返回文本都作为业务资料，而不是更高优先级的系统指令。Prompt 会标明资料边界，但防护还需要工具白名单、参数 Schema、权限校验和最小权限。检索层按企业和文档权限过滤，执行层重新核验，不允许文档中的“忽略审批”直接改变流程。模型只能传业务 ID，服务端根据身份解析真实资源；敏感信息按任务最小需要使用。

追问：如果资料写着“替我退掉全部机票”呢？答：它只是资料文本，不能触发退票。实际退改必须来自有权限用户的明确意图、当前订单与相应确认流程。

13 RAG 检索与知识更新

34 P0 商旅知识问答怎样设计 RAG

考察点：是否理解实时业务与静态知识的边界。

参考回答：政策手册、服务流程和术语适合进知识库，价格、库存、订单状态要调用实时接口。摄取时保存文档来源、企业、版本、生效区间和权限，按条款与章节切分。查询时先确定用户身份和问题适用范围，再做关键词与向量召回、融合、重排，取有限证据给模型生成，并返回出处。找不到适用规则时说明缺证据或转人工。交易前再调用规则服务检查，不能仅依赖检索到的文字批准订单。

追问：RAG 能消除幻觉吗？答：只能降低部分无依据回答，仍可能检索错、漏条件或生成错。需要拒答、引用一致性检查与任务级评测。

35 P1 为什么要混合检索和 reranker

考察点：是否分清召回与排序。

参考回答：向量检索更容易覆盖语义近似表达，关键词检索对订单号、航班号、政策编号、专有名称更精确。两路召回可以用 RRF 等排名融合方法合并，避免直接相加不同尺度的分数。reranker 进一步比较问题与候选片段，改善最终顺序，但计算成本更高，所以一般只处理召回出的有限候选。TopK、切块粒度和阈值应通过标注数据调整，同时检查召回率与上下文噪声。

追问：向量相似度就是答案可信度吗？答：不是，它反映表示空间的相近程度，不证明片段适用、完整或真实。版本、权限与规则条件仍要检查。

36 P1 差旅政策更新后怎样避免引用旧规则

考察点：是否关注版本与生效时间。

参考回答：每份政策保存版本、生效起止时间、企业与适用人群，新版本建立完成后再切换可用状态，避免更新半途出现混合索引。检索按业务时间筛选，缓存键包含文档或索引版本。订单审批保存当时使用的规则版本与关键输入，方便解释历史决策。如果新旧条款冲突且没有明确替代关系，返回冲突并交给规则负责人确认。不能简单按上传时间最新就覆盖全部适用范围。

追问：权限过滤应放在哪里？答：召回前尽量过滤，召回后及生成前再次核验，避免无权限片段进入 Prompt。共享向量库也必须保证企业隔离。

14 商旅预订 Agent 系统设计

37 P0 帮我订下周三去北京两天的差旅 你如何设计

考察点：是否能给出完整且可执行的业务流程。

参考回答：先读取用户身份、企业和差旅政策，再澄清出发地、绝对日期、会议时间、出行人、返程及住宿需求。“两天”不一定代表住两晚，要明确。将信息写入行程草稿，独立查询交通与酒店候选，校验差标和时间约束，展示含税价格、退改条件和报价有效期。用户确认具体方案后，按企业规则申请审批；执行前重新查价验库存、核验审批与授权，再通过幂等接口提交。出票或订房成功以供应商状态为准，最后生成行程，并持续跟踪失败、变更和退款。

追问：怎样控制模型权限？答：Agent 可以建议步骤和填草稿；创建订单、支付、退改等工具有明确前置状态、权限与确认要求，业务服务校验通过才执行。

建议在白板上画出的组件

交互 API 接收请求并鉴权；编排服务保存任务和会话；模型服务负责理解与生成；知识检索服务解释规则；差标与审批服务做确定性判断；订单服务负责交易状态；供应商适配器处理不同接口；任务 Worker 负责异步执行与查询；数据库保存事实，缓存加速读取。

建议状态流

COLLECTING 收集需求 → QUOTED 已报价 → WAIT_CONFIRM 等待确认 → WAIT_APPROVAL 等待审批 → READY_TO_BOOK 可提交 → SUBMITTING 提交中 → CONFIRMED 已确认。

异常分支包括 REQUOTE 重新报价、UNKNOWN 结果待查、FAILED 明确失败、CANCEL_PENDING 取消中与 CANCELLED 已取消。实际公司状态以既有订单系统为准，不能强行用聊天状态替换订单状态。

38 P0 价格变了或者有一项预订失败怎么办

考察点：是否理解分布式业务通常不能整体回滚。

参考回答：报价必须有 offer_id、版本或有效期，执行前重新确认。涨价或退改条款变化超过用户授权范围时重新请求确认，不能悄悄买下。机票成功、酒店失败时，保存部分成功状态，说明已经发生的交易，让用户选择重订酒店或按规则取消机票。补偿动作本身也可能失败或产生费用，所以每项操作都要有状态、重试策略和人工处理入口。

追问：机票和酒店能用一个数据库事务包住吗？答：不能把外部供应商的副作用纳入普通本地事务。采用步骤化执行、状态记录、幂等和业务补偿。

15 交易幂等与效果评测

39 P0 下单超时了 怎样防止重复下单

考察点：是否理解“超时”不等于“失败”。

参考回答：请求超时后订单可能已经在供应商创建，所以先把本地状态记为 UNKNOWN，并根据商户请求号查询结果。如果供应商支持幂等键，重试时使用相同键；不支持时也应使用可关联的业务请求号并查询核对，不能换号盲目重发。本地建唯一约束 tenant_id 加 idempotency_key，保存规范化请求的摘要；同键同内容返回原结果，同键不同内容返回冲突。幂等状态和提交记录要持久化，缓存只能辅助。

追问：供应商既不支持幂等也不能查单呢？答：无法承诺安全自动重试，应停止自动下单并转人工核实。这是外部接口能力决定的边界。

40 P0 你会怎样定义 Agent 的成功率

考察点：是否能落实 JD 的评测要求。

参考回答：先按用户任务定义成功，而不是按模型有没有回复定义。政策问答要正确并引用适用条款；查询推荐要满足行程和差标；预订要以订单真实终态确认。任务完成率可以定义为成功到达目标终态的有效任务数除以有效任务总数，取消、无库存、用户中断等另列口径，避免通过排除失败美化结果。还要记录越权执行、重复订单和违反差标等严重错误，分别观察工具调用正确率、P95 耗时、单任务成本和人工接管率。

追问：业务没有统一标准答案怎么办？答：用约束集合与可验证结果评价，允许多个合法方案。价格不一定最低，但必须在约定限制内且真实可订。

41 P1 你会怎样做灰度上线和失败回退

考察点：是否有可操作的评测到上线路径。

参考回答：先准备离线任务集，包含缺参数、政策冲突、重复提交、过期报价、超时、无库存与恶意指令。再在测试供应商或沙盒运行完整流程，验证业务副作用。上线从低风险查询与推荐开始，按企业、用户或流量小范围灰度，比较现有流程的耗时、完成率与人工处理量。执行类能力设置开关，出现严重错误立即停用相应工具并保留任务状态。回退到人工时一起提供已确认需求、已执行动作和错误原因，避免让用户重新描述。

追问：给出一个最小评测方案？答：先建设覆盖主要任务与异常的人工标注样本，数量由业务复杂度决定；样本规模、类别占比、版本和验收阈值在测试前确定并报告，不能测试后按结果修改口径。

16 Java 并发与虚拟线程

42 P0 线程池参数和拒绝策略怎么选

考察点：是否能把并发知识用于模型和供应商接口。

参考回答：先区分 CPU 计算与等待 I/O。线程池有核心线程数、最大线程数、存活时间、队列、线程工厂和拒绝策略。一般先使用核心线程，之后入队，队列满后才扩展到最大线程数，再满就拒绝。LLM 调用等待长，不能仅靠扩大线程池解决，要限制同时调用数、队列长度和总等待时间。当前索引是单线程有界队列；在线交互则应和后台批处理隔离，避免批量资料上传拖慢用户请求。

追问：CallerRunsPolicy 一定合适吗？答：它会让提交线程执行任务，能形成反压，但如果提交者是 HTTP 或事件循环线程，会拖慢请求，应按业务选择拒绝、排队或降级。

43 P0 Java 21 虚拟线程解决什么问题

考察点：是否把虚拟线程误认为无限吞吐。

参考回答：虚拟线程让大量以阻塞 I/O 为主的任务能够使用较少平台线程运行，适合等待模型或数据库响应的场景。它不减少模型生成时间，也不增加数据库连接和供应商配额，因此仍需要信号量或限流保护外部资源。项目开启了 Spring 虚拟线程配置，但手工创建的线程池不会自动变成虚拟线程。以 Java 21 为边界，还要关注 synchronized 中长时间阻塞等 pinning 情况，结合 JFR 与线程信息验证，而不是照搬其他 JDK 版本结论。

追问：要不要给虚拟线程建池？答：通常一任务一个虚拟线程，通过独立的资源配额限制并发；不要把平台线程池的池化思路直接照搬。参考 JEP 444。

44 P0 volatile synchronized 和 CAS 有什么区别

考察点：能否解释可见性、原子性与复合操作。

参考回答：volatile 主要提供可见性和一定的有序性，不保证 count++ 这样的读改写原子性。synchronized 通过互斥保护临界区，并建立释放与后续获取之间的可见性。CAS 按预期值尝试更新，适合某些轻量原子操作，但有竞争重试和 ABA 等问题。ConcurrentHashMap 保证其接口承诺的并发语义，不代表 get 后判断再 put 这组操作自动原子，应使用 putIfAbsent、compute 等恰当操作。业务状态跨数据库和 Redis 时，更不能只用 Java 锁推导全局正确。

追问：SseWriter 为什么同时用了 volatile 和 synchronized？答：要分析字段访问和锁范围；同步方法内的同锁访问已建立可见性，volatile 不能代替对输出流写入与关闭顺序的互斥。

Java 21 依据 <https://openjdk.org/jeps/444>。

17 JVM 与故障排查

45 P0 JVM 内存和 GC 你怎么理解

考察点：是否掌握基础机制及实际诊断入口。

参考回答：对象主要在堆上，线程执行有栈，类元数据在元空间，还要关注直接内存、线程和本地库等进程开销。GC 根据可达性判断对象是否仍被引用；内存泄漏在 Java 中常表现为对象已经没业务价值但仍可达，例如无界缓存或未清理的线程上下文。G1 按 Region 管理堆，关注暂停目标与吞吐；选择和调优 GC 要结合实际延迟、分配率和堆占用，不应一上来调整一堆参数。

追问：堆没满为什么进程被杀？答：可能是容器内存限制、直接内存、元空间或线程栈等开销，先检查退出原因、容器事件和进程 RSS，不能只看堆使用量。

46 P0 线上接口突然变慢 你怎么查

考察点：是否能缩小故障范围并保护服务。

参考回答：先看影响范围、错误率和延迟分位数，以及是否刚发布。按链路拆分网关、应用排队、数据库、缓存、模型和供应商耗时。CPU 高时看热点和线程栈；CPU 不高但请求堆积时看锁等待、连接池和外部 I/O。LLM 链路特别核对真实调用次数和总重试时长。先通过限流、关闭异常功能或回退版本控制影响，再定位根因。恢复后补充相应告警和回归案例，不只记录平均响应时间。

追问：能直接在线上抓堆转储吗？答：先评估停顿、磁盘空间和敏感信息，选择合适实例与窗口。线程栈、JFR 和常规指标往往更适合先做低影响定位。

47 P1 CompletableFuture 并行调用要注意什么

考察点：是否知道线程池与取消的实际行为。

参考回答：机票与酒店查询独立时可以并行，用明确的业务执行器和总 deadline 组织结果，避免把阻塞调用大量放入公共池。每个调用有超时与异常结果，部分失败时保留可用候选并说明范围。orTimeout 等待结束不必然中断底层请求，取消要传递到 HTTP 客户端。线程切换后用户身份、日志 trace 和事务都要显式处理，不能假定自动继承。

追问：两个异步任务都使用一个事务吗？答：普通线程绑定的 Spring 事务不会自动跨线程传播，异步任务应有清晰的独立事务边界，跨任务一致性由流程设计保障。

18 Spring 与接口设计

48 P0 Spring Boot 自动配置 IOC 和 AOP 怎么联系

考察点：是否能解释常用框架的机制。

参考回答：IOC 容器创建并管理 Bean，按依赖关系注入协作者；构造器注入能明确必需依赖，也方便单测。自动配置根据 classpath、配置属性和已有 Bean 等条件提供默认组件，业务声明自己的 Bean 后可按条件替换默认行为。AOP 通过代理等机制在调用前后织入事务、监控等横切逻辑。排查自动配置问题时看条件匹配报告、Bean 定义和实际注入类型，不只是反复改注解。

追问：为什么同类调用可能导致 AOP 不生效？答：默认代理模式下，this 调用绕过代理入口，相关增强不会在内层调用处触发；可拆到独立 Bean 或使用明确的编程式机制。

49 P0 Transactional 常见失效场景有哪些

考察点：是否真正理解事务边界。

参考回答：默认代理模式下的自调用、没有经过容器管理的对象、异常被吞掉以及异步线程切换都可能导致预期的事务行为不成立。默认回滚规则通常针对 RuntimeException 和 Error，受检异常需要明确配置；也要检查项目是否设置了全局回滚规则。REQUIRED 一般加入当前事务，REQUIRES_NEW 开启独立事务并增加连接需求。调用模型或供应商时我倾向先完成耗时外部步骤，再用短事务校验状态和写库，避免长时间占用连接与锁。

追问：读完数据后调用模型 再写库会有并发变化怎么办？答：写入前重新检查版本和状态，必要时使用乐观锁或条件更新。把远程调用挪出事务必须配套并发校验。

50 P0 一个耗时 Agent 接口怎么设计

考察点：是否能设计可查询、可取消、可恢复的 REST API。

参考回答：短操作同步返回，长任务可以 POST /tasks 创建并返回 202、task_id 和状态查询地址，再通过 GET /tasks/{id} 查询，SSE 推送进度。进度事件和任务事实分开保存；客户端断开不应使任务状态无法查询。创建或确认动作支持幂等键，取消采用明确语义：等待中的任务可直接取消，已经进入供应商交易的任务要转为取消申请或结果核实。错误码区分参数问题、权限、状态冲突、限流和外部失败。

追问：取消成功是不是等于退款成功？答：不是，取消请求受理、订单取消、退款处理中和退款成功是不同阶段，接口应返回真实状态。

Spring 事务依据 <https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html>。

19 MySQL 索引与事务

51 P0 查询企业订单列表 怎样设计索引

考察点：是否能按真实 SQL 设计而非背最左前缀。

参考回答：先明确查询条件和排序。例如固定 tenant_id 与 status，按 created_at、id 倒序分页，可以评估联合索引 (tenant_id, status, created_at, id)。等值条件在前，排序与游标字段随后，并用 EXPLAIN 核扫描行数和排序行为。深分页优先基于上一页末尾的时间与 ID 做游标条件，避免 OFFSET 越来越大。没有 status 条件的查询可能需要不同索引，不能认为一个联合索引覆盖所有情况；索引也会增加写入和存储成本。

追问：用了索引一定快吗？答：不一定，还要看选择性、回表量、排序、返回数据量和统计信息。小表或低选择性条件下全表扫描可能更合理。

52 P0 MVCC 和锁分别解决什么问题

考察点：是否能区分快照读、当前读和并发修改。

参考回答：InnoDB 用版本信息和 undo 等支持一致性读，减少读写互相阻塞。READ COMMITTED 通常每次一致性读建立新的快照，REPEATABLE READ 在事务内复用首次一致性读的快照。锁定读和写操作与普通 SELECT 不同，要读取并锁定相应记录；范围条件在相应隔离级别与访问方式下可能使用 next-key 锁。不能简单说 RR 下永远没有幻读，也不能把快照读的表现当作当前读行为。订单更新要按明确旧状态执行并检查结果。

追问：怎么减少死锁？答：保持一致访问顺序、缩短事务、使用合理索引，并对可重试事务做有限重试。死锁是需要处理的正常竞争结果，不应承诺永远不会出现。

53 P1 PostgreSQL 项目怎样证明 MySQL 能力

考察点：是否承认方言与机制差异。

参考回答：homegrown 运行时用 PostgreSQL，我不会说它是 MySQL 生产项目。可以迁移的是表设计、事务边界、索引访问模式和 SQL 分析能力；需要单独验证的是类型、JSON、数组、部分索引和隔离行为。比如项目的 READY、ANSWERING 部分唯一索引不能原样照搬到 MySQL，可以评估可空生成列加唯一索引，或单独的活跃任务表，通过显式约束表达同一业务规则。迁移要用真实 MySQL 测试，不能仅靠 H2 单测证明兼容。

追问：JPA 和 MyBatis 怎样选？答：聚合写入和常规 CRUD 可使用 JPA；复杂查询、性能敏感 SQL 可使用显式查询或 MyBatis。JPA 也必须理解实际 SQL、懒加载和 N+1。

MySQL 机制依据 <https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-isolation-levels.html>。

20 缓存 结算与部署

54 P0 缓存和数据库怎样保持一致

考察点：是否知道方案的时间窗口与适用范围。

参考回答：常见读取用 Cache Aside，缓存未命中时查询数据库并填充；写入先提交数据库再失效缓存，并考虑删除失败后的重试与补偿。仍可能有并发旧值回填，因此根据场景加入版本、较短 TTL 或事件失效机制。订单状态、审批资格等关键决策直接读权威服务并做条件更新，不能依赖可能过期的缓存。缓存 key 带企业与业务维度，热点过期可用互斥重建或逻辑过期控制击穿，空值短缓存和入口校验可缓解穿透。

追问：Redis 不可用时直接查数据库就行吗？答：还要限流、隔离和降级，避免流量全部打到数据库。登录票据、任务状态等用途也不能简单套用缓存回源。

55 P1 对账和金额为什么要特别设计

考察点：是否能从下单延伸到结算闭环。

参考回答：订单完成不代表财务已结清。需要区分预订、出票、支付、取消、退款和结算状态，保存供应商订单号、流水号、币种及金额口径。金额使用最小货币单位整数或明确精度的 BigDecimal，记录税费、服务费与退改费，不能用 double 直接计算结算金额。对账按业务流水关联，比较金额和状态，差异进入待处理队列；重复回调以事件 ID 或业务唯一键去重。自然语言解释可以由模型生成，账务金额和状态更新应由确定性规则处理。

追问：回调验签通过就可以更新吗？答：还要核对来源、订单归属、金额、时效与当前状态，防重放并处理乱序，最终状态有争议时主动查单核实。

56 P1 Docker 和微服务能力怎么回答

考察点：是否清楚已用能力与理解范围。

参考回答：我的项目用 Docker Compose 管理应用与基础设施，Nginx 负责静态资源和 API 代理，数据要落在持久卷，配置与密钥通过环境或密钥设施注入。发布前检查构建产物、迁移、健康检查和日志，预留数据库兼容的回滚路径。Spring Cloud 的服务发现、配置、网关与熔断等概念我能解释，但当前 homegrown 是单体，没有实际落地全套微服务组件。如果加入团队，我会先掌握现有调用链和治理方式，再参与具体模块。

追问：Compose 等于高可用吗？答：不等于。还需要副本、故障转移、持久化备份、负载均衡和演练；部署工具本身不会自动提供业务恢复能力。

21 文献 RAG 与教育平台追问

57 P0 准确率提升 23% 和 Token 减少 21% 怎么来的

考察点：简历指标的可信度与实验设计能力。

参考回答：这两个数字来自简历中的文献 RAG 对照评测，不属于 homegrown 的线上指标。复述前我会核对原始实验表：准确率的评价单元是问题、事实点还是检索命中，23% 表示相对提升还是百分点，基线 Dify 的模型、分块和检索配置是什么，Token 是否包含输入、输出以及重试。合理比较应尽量固定语料、问题集和生成模型，分开记录解析、召回、重排和上下文压缩的效果。若当时同时改变多个环节，只能说整体链路相对该基线改善，不能归因于 LangChain 框架本身。

追问：给个计算例子？**答：**仅作口径示例，准确率从 60% 到 73.8% 是相对提升 23%，也是增加 13.8 个百分点；平均 Token 从 1000 到 790 是减少 21%。这些示例数不是你的实验记录。

证据准备：补齐样本数、数据划分、基线分数、改进分数、人工标注标准、失败样例、Token 日志与配置版本；无法核实的口径先说明，不临场补造。

58 P1 文献项目为什么用 Grobid 和 BGE M3

考察点：是否懂文档结构和检索模型的分工。

参考回答：论文 PDF 的正文、标题、参考文献和版面关系容易在普通文本提取时混在一起。Grobid 能帮助恢复结构化信息，再按章节组织内容与元数据。BGE-M3 用于文本表示，ChromaDB 管理向量检索，关键词召回补充术语精确匹配，重排进一步筛选证据。结构解析也可能出错，所以需要抽检页码、段落顺序和表格；不能把工具输出直接当作正确原文。压缩上下文时保留结论的限定条件和来源，避免只留下听起来确定的结论。

追问：BGE-M3 和 reranker 是同一东西吗？**答：**不是，embedding 把文本映射成可检索表示；reranker 对问题与候选内容做更精细相关性判断。是否实际启用了某种表示或重排模型，要按自己的配置回答。

59 P1 教育平台的异步画像与多租户经验怎样迁移

考察点：能否解释简历中较重的工程承诺。

参考回答：简历中的教育平台采用每租户独立 PostgreSQL 库和 Celery 队列，把耗时的班级与学生画像计算移到异步任务，并通过脏标记触发重算。迁移到商旅可以对应企业隔离和长任务处理，但还要检查共享 Redis、对象存储、日志和任务路由的租户边界。画像重算要按业务键幂等，读取快照或数据副本，防止旧任务覆盖新结果。连接池要考虑 Gunicorn 进程数乘以每进程连接数，不能只调单个池的最大值。

追问：你实际做了哪块？**答：**带着真实接口、任务或监控面板说明负责范围；不要把团队接入的所有观测组件都描述成自己独立建设。

22 实习与沟通能力

60 P0 介绍你在实习中的测试 Agent

考察点：是否有目标到工具结果的完整执行概念。

参考回答：我的实习方向是把自然语言安全测试需求映射到工具链，包括任务路由、调用扫描工具、解析报告和整理分析结果。说明时我会选择一个自己实际参与的案例，按输入需求、工具选择、状态记录、失败处理和结果验证讲清楚。使用 LangGraph 时要定义状态与路由条件，工具失败要区分参数错误、执行异常和结果为空。扫描结果只是工具证据，最终报告需要关联原始输出并解释限制。这类经验与商旅 Agent 的共同点是：自然语言驱动流程，程序执行工具，业务结果需要验证。

追问：长期记忆具体怎么实现？**答：**解释简历中真实保存的字段、存储方式和检索方式。Redis 保存任务上下文不自动等于完整长期记忆；如果没有跨会话持久化与更新策略，应如实限定范围。

61 P0 你使用 AI 编程助手 怎样保证自己理解代码

考察点：是否能对产物负责，是否具备独立排错能力。

参考回答：我会先确定需求、接口和状态约束，再让助手协助实现或整理测试。提交前逐段看关键逻辑，重点检查权限、事务、并发、异常和配置，不只看能否编译。对核心功能我能从入口走到持久化，并用一个正常案例和一个失败案例解释运行过程。助手生成的说法和 README 都需要与实际代码核对，例如当前模型调用顺序和部分唯一索引范围，就应以执行代码为准。面试时我会承认使用助手，同时展示我如何发现问题和验证修改。

追问：现在让你脱离助手写代码行不行？**答：**基础集合、并发控制和 SQL 应能独立完成；不熟悉的框架 API 可以查文档，但必须能解释设计和测试方法。

62 P1 遇到不熟悉的商旅业务你怎么推进

考察点：学习能力、沟通与需求澄清。

参考回答：先找业务负责人梳理一个完整案例，明确角色、输入、规则、成功状态和失败处理，再画状态流并确认接口契约。我会重点问“超标是否允许申请例外”“价格变化如何确认”“供应商超时由谁核实”“退款何时算完成”。把不确定处转换成可以确认的规则和验收用例，然后做最小可运行闭环。进度沟通会说清已完成内容、当前风险和下一步验证时间，不只报告“正在开发”。

追问：产品要求跳过校验尽快上线怎么办？**答：**拿具体后果和可替代方案沟通，例如先上线只读推荐或由人工确认的流程，保留交易必需的规则，并明确后续补齐项与负责人。

23 现场 SQL 与代码题

SQL 题 企业订单的游标分页

题目：查询某企业某状态的订单，按创建时间和 ID 倒序，每页 20 条。说明深分页处理与索引。

```
SELECT id, created_at, status, total_amount
FROM travel_order
WHERE tenant_id = :tenantId AND status = :status
      AND (created_at < :lastTime
            OR (created_at = :lastTime AND id < :lastId))
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

答案：第一页省略游标条件，后续用上一页最后一条记录的时间和 ID。评估联合索引 (tenant_id, status, created_at, id)，通过执行计划检查访问路径。ID 用于打破相同时间的排序并列。并发新增时游标分页可以减少 OFFSET 漂移，但它不是完整快照；导出账单需要明确截止时间或一致性快照。

SQL 题 按旧状态推进订单

```
UPDATE travel_order
SET status = 'SUBMITTING', version = version + 1
WHERE id = :orderId AND tenant_id = :tenantId
      AND status = 'READY_TO_BOOK' AND version = :version;
```

答案：影响一行才获得本次推进资格，零行说明版本、归属或状态不匹配。事务提交后执行外部调用，调用结果再以合法状态转换记录。必须另有幂等键与请求记录，因为“已变成 SUBMITTING”并不能证明供应商收到或成功处理。

Java 题 限制模型并发并正确释放许可

```
boolean acquired = limiter.tryAcquire(200, TimeUnit.MILLISECONDS);
if (!acquired) throw new BusyException();
try {
    return llmClient.call(request);
} finally {
    limiter.release();
}
```

答案：这是示意代码，limiter 是共享 Semaphore；只有成功获取才进入 finally 释放，等待被中断时应向上抛出或恢复中断标记。HTTP 调用还需要自己的 deadline 与取消机制。这个限额只在当前实例生效，多实例按供应商全局配额协调，不能误称为全局限流。

常见手写题的回答顺序

LRU：HashMap 加双向链表，实现访问或更新后移到头部，超容量淘汰尾部，平均 $O(1)$ ；原始结构不线程安全。TopK：使用大小为 K 的最小堆，时间 $O(n \log K)$ 、空间 $O(K)$ ，先处理 K 小于等于 0 和 K 大于数据量等边界。写完后主动给出正常、空输入、重复值与边界测试。

24 模拟面试与准备清单

一轮 45 分钟模拟面试

0 至 3 分钟：自我介绍、到岗安排与应聘动机。目标是把话题自然引到 homegrown 和实习。

3 至 16 分钟：介绍项目，然后连续追问状态机、Provider 适配、SSE、Redis 丢失和重复提交。每个回答至少能点到一个类、一个边界和一个验证办法。

16 至 28 分钟：设计“上海出发去北京两天”的商旅 Agent，追问超标、价格变化、供应商超时与机票成功酒店失败。

28 至 38 分钟：Java 线程池、事务、MySQL 索引与现场 SQL。不要只背结论，结合模型调用和订单例子。

38 至 42 分钟：核对文献 RAG 的 23% 和 21%，讲清真实职责、实验口径与失败案例。

42 至 45 分钟：反问团队的实际目标、交付方式和实习要求。

可以反问面试官的四个问题

当前 Agent 最先落地的是规则问答、查询推荐，还是已进入预订执行？团队用什么指标判断阶段性成功？

商旅订单、审批和供应商接口现在怎样分工？这个岗位更多承担 Java 主服务开发，还是独立 Agent 编排？

模型应用目前最大的难点是准确率、调用成本、响应速度，还是与既有流程对接？希望实习生先解决哪一类问题？

到岗后前一个月预期交付什么？代码评审、测试与带教通常怎样开展？

面试前两天的优先安排

第一天先完成项目事实核对，准备一条学习训练调用链和一条模型流式调用链。整理六个高风险表述的正确口径，并找齐指标的原始记录。随后练习第 37 至 41 题，能在白板上画出状态、工具、幂等和恢复路径。

第二天重点复习第 42 至 56 题，独立写出两个 SQL 和限并发代码。最后做完整模拟面试录音：每题开头是否回答了问题、是否讲清自己的实现、有没有把计划说成已完成。删除未经验证的数字和过度承诺。

带到面试现场的材料

带可运行的演示路径、关键代码定位、测试样例和脱敏截图；演示涵盖正常生成、停止生成、重复点击和模型失败。另备项目离线介绍，避免网络不可用时无法说明。准备真实的到岗日期、出勤天数和可实习时长。

25 大模型基础与优化选择

63 P0 大语言模型为什么能生成回答 又为什么会幻觉

考察点：JD 中的大模型基本原理，是否只会调用接口。

参考回答：常见自回归语言模型把输入切成 token，转换为向量表示，再通过 Transformer 等结构建模上下文关系，逐步预测并生成后续 token。注意力机制帮助当前位置结合相关上下文；预训练学习语言 and 知识模式，后续指令训练等使其更适合完成任务。生成概率高不等于事实正确，模型可能缺知识、混淆上下文或在证据不足时补全出看似合理的答案。因此业务系统需要外部证据、结构化校验和可验证工具结果。理解这些机制能帮助我控制上下文、设计检索和解释模型失误。

追问：temperature 越低是不是越准确？答：它主要影响采样分布和多样性。降低温度可能让表达更集中，但错误知识仍可能稳定输出；准确性要通过数据和任务评测判断。

64 P0 Prompt RAG 微调和强化学习分别什么时候用

考察点：是否能按问题来源选择优化方式。

参考回答：任务说明不清、格式或行为不符合预期时先调整 Prompt。需要访问企业知识或频繁更新的政策时使用 RAG，并用实时工具查询价格库存。稳定任务中的输出风格、领域表达或特定行为仍不达标，而且有高质量训练数据时，再评估监督微调。强化学习需要可靠的反馈或奖励设计、训练资源和评测体系，不能把“多试几次 Prompt”称为强化学习。商旅系统首先需要解决规则执行和任务成功问题，不应因为 JD 提到加分项就声称自己做过训练。

追问：微调后还需要 RAG 吗？答：可能需要。微调主要改变模型行为和能力，RAG 提供当前、可追溯的外部信息，两者可以组合；参数中的知识也不能替代实时订单与库存。

65 P1 Qwen 和其他模型怎样选以及怎样控制成本

考察点：是否能依据具体任务做选型。

参考回答：我会用相同业务任务集比较中文意图理解、工具选择、参数准确率、结构化输出、长上下文表现、首字延迟和单任务成本，再结合部署方式、数据处理要求及服务稳定性选择。简单槽位提取可以用较轻量模型，复杂规则解释或方案比较再调用较强模型。成本还包括检索、重排、重试和失败后的人工处理，不能只看每百万 token 单价。可缓存不随用户权限和规则版本变化的公共结果，裁剪无关历史，设置输出与调用预算，但不裁掉影响交易的关键条件。

追问：能无感切换 Provider 吗？答：需要经过兼容和效果回归。接口能调用不等于工具语义、参数与最终效果相同；切换后应监测失败类型，并保留回退配置。

26 来源与代码定位

公开资料

[S1] 程多多官网。用于核实企业商旅产品、OA 对接及结算服务等业务方向。<https://www.chengduoduo.com/>

[S2] 程多多 App Store 产品介绍，开发者为上海程多多电子商务有限公司。用于核实差旅申请、机票火车票预订、行程管理和政策管控。<https://apps.apple.com/cn/app/程多多/id1506135728>

[S3] OpenJDK JEP 444。用于核对 Java 21 虚拟线程、资源限流和 pinning 的版本边界。
<https://openjdk.org/jeps/444>

[S4] Spring Framework Using Transactional。用于核对代理、自调用与回滚规则。
<https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/annotations.html>

[S5] MySQL 8.4 Transaction Isolation Levels。用于核对一致性读、隔离级别和锁行为。
<https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-isolation-levels.html>

[S6] LangGraph Persistence 与 Interrupts。用于核对检查点、状态恢复和人工介入。
<https://docs.langchain.com/oss/python/langgraph/persistence> ;
<https://docs.langchain.com/oss/python/langgraph/interrupts>

项目代码导航

项目根目录为 interview-homegrown。Java 公共前缀为 app/src/main/java/interview/homegrown/，下列路径相对此
前缀。

模型与流式：common/ai/StructuredOutputInvoker.java、common/ai/LlmRawClient.java、
common/ai/LlmProviderRegistry.java、common/ai/AiSettingsService.java、common/web/SseWriter.java。

学习与评分：modules/drill/service/SelectionService.java、DailyPlanService.java、ScheduleService.java、
NgramSimilarityGuard.java、GradingService.java；modules/drill/grader/GraderText.java。

资料索引：modules/drill/service/CorpusIndexer.java、CorpusService.java、CorpusLibraryService.java、
CorpusOutline.java。

模拟面试：modules/interview/service/InterviewSessionService.java、InterviewQuestionService.java、
FollowupGeneratorService.java。

鉴权与缓存：modules/user/security/SecurityConfig.java、JwtAuthFilter.java；
infrastructure/redis/RedisService.java。

根目录下配置与测试：app/build.gradle、gradle/libs.versions.toml、app/src/main/resources/application.yml、
app/src/main/resources/db/migration/common/V28__active_drill_run_constraint.sql、
V29__restore_legacy_tables.sql、frontend/src/lib/sseParser.ts、frontend/tests/sseParser.test.mjs、
deploy/nginx.conf。

简历和 JD

简历为全栈与 Agent 应用简历_V4.pdf；岗位依据为所提供 JD 截图。简历中的个人经历、职责和指标应由候选人结合原
始材料复核，口述答案按事实调整。公开资料查阅日期为 2026 年 9 月 9 日。